SomnGuard: sistema edge–cloud para el monitoreo de somnolencia y fatiga durante la conducción

Cristian Palma
SENA – Servicio Nacional de Aprendizaje
Neiva
Colombia

Juan Carlos Jurado
SENA – Servicio Nacional de Aprendizaje
Neiva
Colombia

Johan Rodriguez
SENA – Servicio Nacional de Aprendizaje
Neiva
Colombia

Brayan Perdomo
SENA – Servicio Nacional de Aprendizaje
Neiva
Colombia

Resumen

SomnGuard es un sistema edge–cloud orientado al monitoreo de somnolencia, fatiga y eventos de distracción durante la conducción. El sistema combina un dispositivo con cámara y procesamiento local, una API para la gestión de eventos y dispositivos, una base de datos PostgreSQL, una aplicación móvil y un portal web. Mediante un estudio de caso con desarrollo iterativo Scrum, este artículo presenta el diseño e implementación del sistema, describe la operación offline y la sincronización posterior de eventos, y reporta una evaluación preliminar basada en pruebas automatizadas, validación de migraciones y evidencia funcional del dispositivo y la aplicación. Los resultados muestran que existe una implementación operativa de los flujos principales, aunque todavía no se dispone de una evaluación completa en condiciones reales de conducción ni de métricas consolidadas de precisión, cobertura o rendimiento. Se discuten decisiones arquitectónicas, amenazas a la validez y limitaciones, y se derivan lecciones aprendidas y requisitos para una futura validación de campo con un protocolo reproducible.

Keywords

somnolencia al volante, edge computing, visión por computador, monitoreo de conductores

1. Introducción

La somnolencia y la fatiga durante la conducción reducen la atención y pueden afectar la capacidad de respuesta del conductor. El problema es especialmente relevante en trayectos prolongados, turnos nocturnos y actividades de transporte en las que un episodio breve de pérdida de vigilancia puede producir consecuencias graves. Aunque existen soluciones comerciales y experimentales de asistencia al conductor, una solución útil debe considerar no solo la detección visual, sino también la generación de alertas, la continuidad operativa, el registro de eventos y la consulta posterior de la información. La literatura de ingeniería muestra que prácticas como la integración continua [Fowler and Foemmel 2006], el desarrollo dirigido por pruebas [Beck 2003] y la atención a la calidad del código [Martin 2008] mejoran la verificación de este tipo de sistemas, aunque su efectividad depende del contexto.

SomnGuard aborda este problema mediante una arquitectura edge–cloud. Un dispositivo con cámara procesa localmente señales visuales asociadas con somnolencia y distracción, genera alertas cercanas al conductor y conserva eventos cuando no hay conectividad. La plataforma central recibe y almacena los eventos sincronizados, mientras que la aplicación móvil y el portal web permiten consultar el historial, revisar alertas y gestionar dispositivos. El procesamiento en el borde sigue los principios de edge computing de acercar el cómputo a la fuente de datos para reducir la dependencia de la red [Shi et~al. 2016].

El objetivo de este trabajo es describir la construcción de SomnGuard y aportar evidencia preliminar de su funcionamiento. La pregunta de investigación que guía el estudio es: ¿puede una arquitectura edge–cloud modular sostener la detección local con alerta inmediata, la operación offline con sincronización posterior y la gestión multiplataforma de eventos? El artículo no presenta una validación clínica ni afirma una precisión específica del detector; distingue entre la implementación de funcionalidades, la verificación automatizada de componentes y la validación de campo que aún debe realizarse.

Las contribuciones son: (1) la descripción de una solución edge–cloud modular para monitoreo de eventos de conducción; (2) la documentación del flujo de detección local, sincronización offline y consulta multiplataforma; (3) la sistematización de pruebas automatizadas y validaciones de base de datos disponibles con un protocolo reproducible; y (4) la identificación explícita de limitaciones, amenazas a la validez y requisitos para una evaluación experimental posterior, de la que se derivan lecciones aprendidas transferibles.

2. Marco teórico y trabajos relacionados

2.1. Somnolencia y conducción

La somnolencia al volante constituye un problema de seguridad porque puede disminuir la atención, prolongar el tiempo de reacción y producir episodios breves de pérdida de vigilancia. Entre los indicadores observables se encuentran el cierre prolongado de los ojos, el parpadeo anómalo, el cabeceo y la inclinación de la cabeza. Los métodos basados en señales visuales son especialmente útiles cuando se requiere una supervisión no invasiva, aunque su desempeño depende de la iluminación, la posición de la cámara, las oclusiones y las características del conductor [Dinges and Grace 1998].

2.2. Visión por computador y procesamiento en el borde

Los sistemas de asistencia al conductor pueden analizar imágenes de una cámara para generar eventos de somnolencia o distracción. SomnGuard adopta este enfoque en un dispositivo edge, de modo que la inferencia y la alerta local no dependen completamente de la conectividad con el servidor. Esta decisión se relaciona con los principios de edge computing, que proponen acercar el procesamiento a la fuente de los datos para reducir dependencia de la red y mejorar la capacidad de respuesta [Shi et~al. 2016].

El procesamiento local no elimina la necesidad de una plataforma central. Por ello, SomnGuard combina detección en el dispositivo con sincronización de eventos, almacenamiento de evidencia y consulta desde una aplicación móvil y un portal web. Esta separación permite que el dispositivo continúe operando ante interrupciones de conectividad y que los eventos puedan enviarse posteriormente al backend mediante un flujo de sincronización idempotente.

2.3. Diferencia frente a enfoques exclusivamente centralizados

Un enfoque exclusivamente centralizado requiere transmitir continuamente datos del sensor o del video, lo que puede aumentar el consumo de red, la latencia y la exposición de información sensible. SomnGuard limita este intercambio a eventos, metadatos y evidencia asociada, manteniendo la detección y la alerta inmediata cerca del conductor. La arquitectura no pretende sustituir una validación clínica ni establecer por sí sola la exactitud de un modelo; ofrece una plataforma de monitoreo y gestión cuya efectividad debe evaluarse posteriormente con pruebas controladas y de campo.

2.4. Prácticas de ingeniería de software

La construcción de un sistema distribuido de cinco componentes exige prácticas de ingeniería verificables. La integración continua reduce riesgos de integración y acelera la detección de errores [Fowler and Foemmel 2006], mientras que el desarrollo dirigido por pruebas mejora la calidad y reduce defectos [Beck 2003]. Los marcos de DevOps y entrega continua vinculan la automatización con el rendimiento del equipo y advierten sobre sus desafíos de adopción [Chen 2015; Forsgren et~al. 2018], y la literatura sobre código limpio enfatiza legibilidad y mantenibilidad como condiciones de evolución [Martin 2008]. SomnGuard adopta estas prácticas de forma selectiva: pruebas automatizadas por dominio en la API, pipelines de integración continua y migraciones versionadas con rollback.

2.5. Vacíos y posicionamiento

Persisten vacíos en trabajos que integren en una misma solución el dispositivo de detección, la API, la persistencia, la aplicación móvil y el portal web con operación offline idempotente y verificación documentada por capa. Como advierten Fitzpatrick y Storey [Fitzpatrick and Storey 2017], aplicar prácticas sin considerar el contexto produce resultados limitados; por ello este trabajo delimita su contexto (prototipo funcional con verificación de componentes) y propone un protocolo reproducible para la validación futura.

2.6. Aporte de SomnGuard

El aporte del proyecto es integrar en una misma solución el dispositivo de detección, la API, la persistencia, la aplicación móvil y el portal web. La documentación del sistema define además mecanismos de operación offline, autenticación, gestión de dispositivos, consulta histórica y notificaciones. En consecuencia, el trabajo se presenta como un diseño e implementación de un sistema edge–cloud con evaluación preliminar, no como un estudio experimental que ya haya demostrado una tasa de precisión o una reducción de accidentes.

3. Metodología

3.1. Diseño del estudio

Se realizó un estudio de caso de investigación aplicada sobre una implementación existente, con ciclos iterativos de planificar–actuar–observar–reflexionar. El trabajo se organizó mediante Scrum, con sprints semanales, historias de usuario, revisión de avances y priorización del backlog. La metodología permitió iterar sobre el dispositivo, el backend, la base de datos y las interfaces sin separar la construcción del producto de la validación progresiva de sus componentes. Se definieron como criterios de éxito: (1) flujos principales operativos extremo a borde; (2) verificación automatizada disponible por componente crítico; y (3) evidencia funcional observable y trazable a ramas de referencia.

3.2. Levantamiento de requisitos

Los requisitos se obtuvieron a partir de la documentación del proyecto, una encuesta sobre seguridad vial y fatiga del conductor, entrevistas a conductores y sesiones de revisión del equipo. Estos insumos se tradujeron en funciones como detección de eventos, alertas, gestión de dispositivos, consulta histórica, operación offline y sincronización.

3.3. Datos e instrumentos

La tabla 1 resume los componentes, tecnologías y ramas de referencia que constituyen las fuentes primarias. Se recolectaron: código y pruebas en `somnguard-*`, ejecuciones de pipelines CI, scripts de migración y rollback de Liquibase, registros del dispositivo y capturas de la aplicación. Los instrumentos corresponden a los propios repositorios y a los flujos de GitHub Actions, cuyas salidas se conservan como evidencia de verificación.

Tabla 1: Componentes de SomnGuard: tecnología y evidencia de verificación disponible

Componente	Tecnología	Rama ref.	Verificación observ
`somnguard-device`	Python + cámara (edge)	`develop`	Logs con eventos etic
`somnguard-api`	Java 21, Spring Boot, Maven	`main`	Pruebas automatizad
`somnguard-db`	PostgreSQL + Liquibase	`main`	Migraciones + rollbac
`somnguard-app`	React Native + Expo	`develop`	Navegación e historia
`somnguard-portal`	React + TypeScript + Vite	`develop`	Consulta y gestión w

3.4. Procedimiento técnico y validez

El procedimiento comprendió cuatro actividades: (1) diseño de la arquitectura edge–cloud y de los contratos de comunicación; (2) implementación independiente del dispositivo, API, base de datos, aplicación y portal; (3) integración mediante eventos, autenticación y persistencia; y (4) verificación mediante pruebas automatizadas, pipelines de integración continua [Fowler and Foemmel 2006], validación de migraciones y observación funcional de los flujos implementados.

La validez se analiza en cuatro dimensiones. La validez interna se apoya en pruebas por dominio y migraciones versionadas, aunque sin medición consolidada de cobertura. La validez de constructo distingue verificación de software (lo que las pruebas realmente comprueban) de validación de detección (lo que requeriría verdad de referencia). La validez de conclusión se limita a afirmar existencia y operatividad de flujos, no eficacia; como advierten Fitzpatrick y Storey [Fitzpatrick and Storey 2017], los resultados no deben generalizarse fuera de su contexto. Por ello, la validez externa se considera preliminar y se reserva una evaluación controlada con conductores y escenarios reales.

4. Implementación del sistema

4.1. Arquitectura edge–cloud

SomnGuard se implementa como un conjunto de componentes coordinados. El dispositivo edge utiliza Python, una cámara y procesamiento local para identificar señales visuales asociadas con somnolencia o distracción. La API, desarrollada con Java y Spring Boot, centraliza autenticación, usuarios, dispositivos, telemetría, eventos, evidencia y notificaciones. PostgreSQL conserva los datos estructurados y Liquibase gestiona las migraciones. La aplicación móvil se construye con React Native y Expo, y el portal web con React, TypeScript y Vite.

4.2. Detección y operación offline

El dispositivo genera eventos locales como parpadeo elevado, ojos cerrados y cabeceo o inclinación. La alerta inmediata se produce en el borde, mientras que los eventos se almacenan temporalmente cuando el enlace con el backend no está disponible. Al recuperar conectividad, el dispositivo sincroniza lotes de eventos y, cuando corresponde, carga la evidencia multimedia. La idempotencia evita duplicados cuando una transmisión debe reintentarse.

4.3. Seguridad y persistencia

La API utiliza JWT con claves RS256 y control de acceso basado en roles para los usuarios. Los dispositivos se autentican mediante API Keys almacenadas de forma segura y sujetas a rotación. La base de datos separa cambios de esquema, datos iniciales, permisos y rollback. La evidencia multimedia se gestiona fuera de PostgreSQL en almacenamiento compatible con S3/MinIO, manteniendo en la base las referencias necesarias para su consulta.

La tabla 2 sintetiza las decisiones arquitectónicas principales frente a sus alternativas. La selección priorizó la continuidad operativa y la mantenibilidad del código [Martin 2008], siguiendo prácticas de automatización y entrega descritas en la literatura DevOps [Chen 2015; Forsgren et~al. 2018].

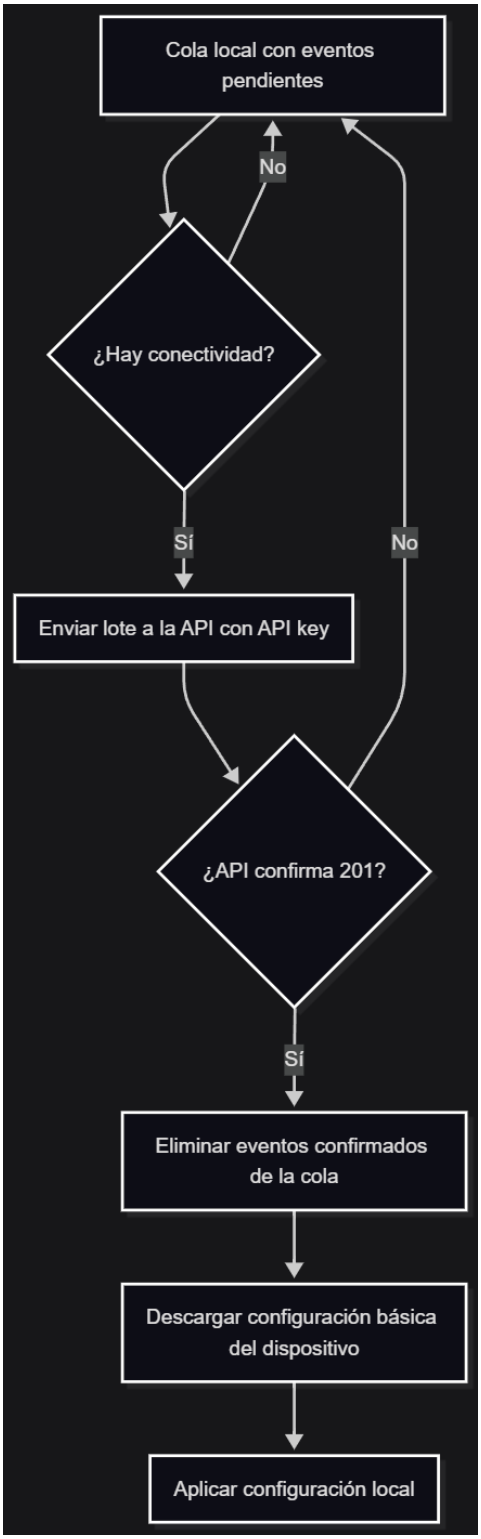


Figura 1: Flujo documentado de sincronización offline del dispositivo.

Tabla 2: Decisiones arquitectónicas principales y alternativas descartadas

Decisión	Alternativa	Motivo
Inferencia local en edge	Inferencia en la nube	Alerta inmediata sin red; m
Sincronización idempotente	Reenvío simple	Evita duplicados ante reinte
PostgreSQL + Liquibase	Esquema manual	Migraciones versionadas y r
Evidencia en S3/MinIO	BLOB en PostgreSQL	Separa datos estructurados
JWT RS256 + RBAC / API Keys	Credencial única	Separa usuarios y dispositiv

4.4. Prácticas aplicadas y pipelines

4.5. Prácticas aplicadas y pipelines

Las prácticas de ingeniería se aplicaron por capa y se verifican en los repositorios de referencia: pruebas automatizadas por dominio en la API [Beck 2003], integración continua con ejecuciones registradas en GitHub Actions [Fowler and Foemmel 2006], migraciones versionadas con Liquibase y scripts de rollback, y observación funcional del dispositivo y la aplicación. Este conjunto reduce riesgos de regresión en backend y base de datos y deja trazabilidad entre el código, la rama de referencia y la evidencia presentada.

4.6. Evidencia funcional del dispositivo

La figura 2 muestra un registro del dispositivo en el que se observan eventos de parpadeo elevado y ojos cerrados. Esta evidencia confirma que el pipeline local puede producir eventos etiquetados; no representa una medición estadística de exactitud.

4.7. Aplicación móvil

Las figuras 3a to 3c muestran tres vistas de la aplicación móvil: inicio, monitoreo e historial. En conjunto, evidencian la navegación principal, el estado del dispositivo, las alertas del día y la consulta de eventos con severidad y evidencia asociada.

5. Evaluación y resultados

5.1. Verificación automatizada

La API cuenta con pruebas para funcionalidades de autenticación, gestión de dispositivos, telemetría, evidencia, consultas, notificaciones, configuración y provisionamiento. Las ejecuciones revisadas de los flujos de integración continua finalizaron satisfactoriamente. En la base de datos, el pipeline de Liquibase valida la aplicación de migraciones y los scripts asociados incluyen operaciones de rollback. Estos resultados constituyen evidencia de verificación de componentes, no una medición de la eficacia del sistema en carretera. La tabla 3 resume el estado por capa.

5.2. Resultados funcionales observables

El registro del dispositivo muestra la producción de eventos identificados como parpadeo elevado y ojos cerrados. La aplicación móvil permite iniciar el monitoreo, consultar el estado de la cámara y del dispositivo, pausar o reanudar la detección, visualizar alertas y consultar el historial. Las capturas presentadas en la sección de implementación documentan estos flujos en una ejecución concreta.

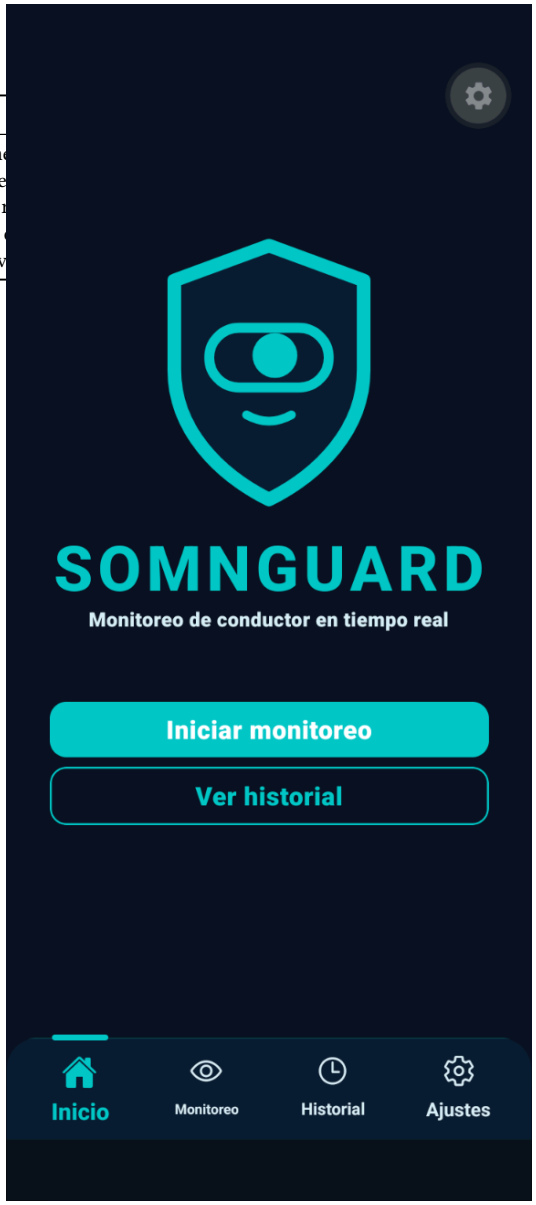


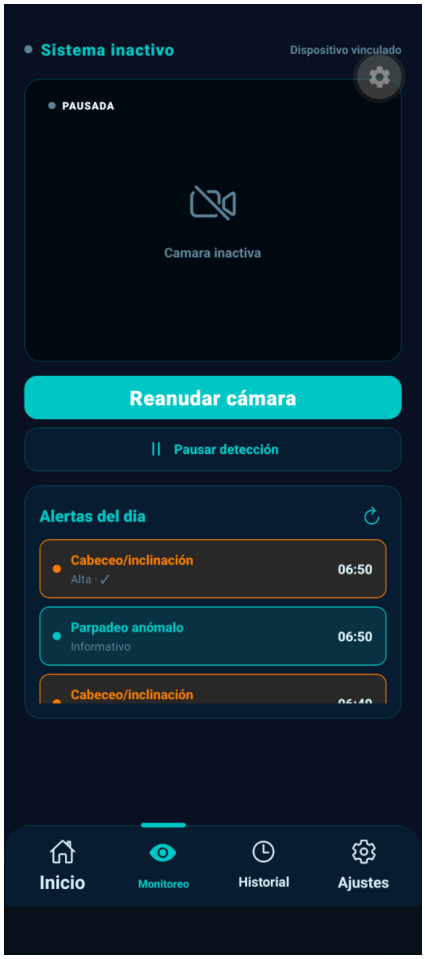
Figura 2: Registro del dispositivo con eventos visuales detectados.

Tabla 3: Verificación por capa: tipo de prueba y alcance observado

Capa	Tipo de verificación	Estado observado
API	Pruebas por dominio + CI	Flujos en verde; sin cobertura consolidad
Base de datos	Migraciones + rollback	Aplicación y reversión validadas
Dispositivo	Logs funcionales	Eventos generados; sin medición estadíst
App móvil	Recorrido funcional	Inicio, monitoreo e historial navegables
Integración E2E	Pendiente	No medida; trabajo futuro



(a) Inicio.



(b) Monitoreo.



5.3. Interpretación

Siguiendo la advertencia de no generalizar prácticas fuera de su contexto [Fitzpatrick and Storey 2017], estos resultados demuestran que los flujos principales están implementados y que las capas críticas disponen de verificación automatizada y trazable. No demuestran exactitud del detector, cobertura total, latencia acotada ni disponibilidad operacional: la presencia de eventos en un log puntual no implica que todos los eventos reales sean detectados ni que las alertas sean suficientes. No se presentan intervalos de confianza porque no existe aún una muestra de campo que los sustente.

5.4. Alcance de los resultados

Los resultados disponibles permiten afirmar que los principales flujos de software están implementados y que algunos componentes críticos tienen verificación automatizada. No se reportan porcentajes de cobertura, precisión del detector, falsos positivos, latencia, disponibilidad ni reducción de accidentes porque no existe todavía un protocolo homogéneo que haya medido esas variables para todo el sistema.

5.5. Evaluación pendiente

La siguiente fase debe incluir un conjunto de conductores, recorridos controlados y condiciones variables de iluminación y conectividad. También debe definir una verdad de referencia para cada evento, comparar las detecciones con observaciones humanas y medir precisión, sensibilidad, especificidad, falsos positivos, tiempo de alerta, continuidad offline y consumo de recursos. La tabla 4 define el protocolo de métricas previsto, incluyendo métricas de proceso DORA [Forsgren 2021; Forsgren et~al. 2018] para la evolución del producto.

Tabla 4: Protocolo de métricas futuras para validación de campo (sin mediciones aún)

Dimensión	Métrica	Método previsto
Detector	Precisión, sensibilidad, especificidad	Comparación contra verdad de ref
Robustez	Falsos positivos por hora	Recorridos controlados con ilumin
Tiempo real	Latencia detección–alerta	Medición extremo a extremo en di
Continuidad	Tasa de sincronización offline	Cortes controlados + reintentos
Producto	Frecuencia, lead time, fallos, MTTR	Métricas DORA en pipelines CI/C

6. Discusión

La principal contribución de SomnGuard es tratar la detección de somnolencia como un problema de sistema y no únicamente como un clasificador de imágenes. La alerta local reduce la dependencia de la red, mientras que la API, la persistencia y las interfaces permiten administrar dispositivos y revisar el contexto de los eventos. La operación offline introduce complejidad de almacenamiento temporal, reintentos e idempotencia, pero es adecuada para escenarios donde la conectividad puede ser intermitente.

La separación entre dispositivo, backend, base de datos, aplicación y portal facilita la evolución independiente de cada componente. Al mismo tiempo, aumenta el número de contratos que deben mantenerse compatibles: autenticación, eventos, evidencia, estados de dispositivo y notificaciones. Las pruebas automatizadas del backend y las validaciones de base de datos reducen riesgos de regresión en esas capas, pero no sustituyen la verificación integrada del flujo completo.

Las capturas y logs aportan evidencia de funcionamiento observable, aunque tienen un alcance limitado: corresponden a ejecuciones puntuales y no permiten generalizar el comportamiento a todos los conductores, vehículos, cámaras o condiciones ambientales. En particular, la presencia de eventos detectados no demuestra que todos los eventos reales sean identificados ni que las alertas sean clínicamente suficientes.

Por tanto, el estado actual del proyecto es compatible con una evaluación preliminar de diseño e implementación. La generalización requiere pruebas de campo, definición de métricas, registro reproducible de condiciones y comparación contra una referencia independiente. Esta delimitación evita confundir la disponibilidad de una funcionalidad con la validación de su efectividad.

La modularidad en cinco componentes tiene un costo: cada contrato (autenticación, eventos, evidencia, estados, notificaciones) debe mantenerse compatible y versionarse, lo que aumenta el esfuerzo de integración frente a un monolito. El beneficio es la evolución independiente y la continuidad offline del borde. Esta compensación entre costo de integración y resiliencia operativa es característica de la entrega continua en sistemas distribuidos [Chen 2015; Forsgren et~al. 2018].

En comparación con la literatura, SomnGuard se alinea con los principios de edge computing [Shi et~al. 2016] y con las prácticas de integración y pruebas continuas [Beck 2003; Fowler and Foemmel 2006], pero se diferencia por integrar detección, sincronización idempotente y gestión multiplataforma en un mismo flujo trazable. Para equipos y decisores, la implicación práctica es que el sistema actual es apto como plataforma de monitoreo verificada a nivel de software, no como detector validado: su adopción operativa exige completar el protocolo de la tabla 4, anonimizar evidencia sensible y registrar condiciones de ejecución para permitir replicación.

7. Conclusiones y trabajo futuro

SomnGuard implementa una solución edge–cloud para monitorear eventos asociados con somnolencia y distracción durante la conducción. El sistema integra detección local, alertas, operación offline, sincronización, persistencia, aplicación móvil y portal web. La evidencia revisada muestra eventos generados por el dispositivo, pantallas funcionales de la aplicación y validaciones automatizadas en la API y la base de datos.

El trabajo no permite afirmar todavía una precisión determinada ni una validación completa en condiciones reales. Esta es una limitación explícita del estudio y no debe ocultarse detrás de resultados de pruebas de software. El siguiente paso consiste en ejecutar una evaluación controlada y de campo que mida el desempeño del detector, la calidad de las alertas, la continuidad offline y la experiencia de los usuarios.

Como trabajo futuro se propone: (1) definir un protocolo de etiquetado y verdad de referencia; (2) evaluar el sistema con distintos conductores, cámaras, iluminación y conectividad; (3) consolidar métricas de cobertura y rendimiento por repositorio; (4) realizar pruebas integradas extremo a extremo; y (5) analizar privacidad, seguridad y aceptación del sistema antes de un despliegue operativo amplio.

La lección principal es separar la verificación de software de la validación de campo: las pruebas automatizadas y las migraciones versionadas [Beck 2003; Fowler and Foemmel 2006] garantizan regresión controlada, pero solo un estudio con conductores reales puede establecer efectividad. Se liberan las ramas de referencia, los scripts de migración y la evidencia funcional descritos en el apéndice como guía reproducible para replicar la verificación y transferir el sistema a nuevos equipos.

A. Checklist de reproducibilidad

- **Datos**: logs del dispositivo y capturas en `graphics/` con condiciones de ejecución anotadas; sin datos personales identificables.
- **Código**: repositorios `somnguard-api`, `somnguard-db`, `somnguard-device`, `somnguard-app` y `somnguard-portal`; ramas de referencia: API y base de datos en `main`, resto en `develop`; registrar el `commit hash` consultado en cada repositorio.
- **Entorno**: API con Java 21, Spring Boot y Maven; base de datos PostgreSQL y Liquibase; dispositivo con Python; interfaces con React/React Native y TypeScript/Vite; versiones exactas y semillas según archivo de cada repositorio.
- **Procedimiento**: (1) clonar ramas de referencia; (2) ejecutar pruebas de cada repositorio; (3) reproducir flujos de GitHub Actions; (4) validar migraciones y rollback de Liquibase antes de integrar; (5) conservar logs y salidas generadas en `build/`.
- **Resultados**: tablas tablas 1 to 4 y figuras `fig:offline_sync`, `fig:device_logs`, `fig:app_home–fig:app_history` generadas a partir de la evidencia disponible.
- **Limitación**: los resultados de este artículo no sustituyen una prueba de campo ni contienen métricas inventadas de precisión, cobertura o rendimiento.

Referencias

Kent Beck. 2003. *Test-Driven Development: By Example.* Addison-Wesley Professional.

Lianping Chen. 2015. Continuous Delivery: Huge Benefits, but Challenges Too. In *IEEE Software*, Vol.~32. 50–54. doi:10.1109/MS.2015.27

David~F. Dinges and Robert Grace. 1998. *Evaluation of Techniques for Ocular Measurement as an Index of Fatigue and as the Basis for Alertness Management.* Technical Report. National Highway Traffic Safety Administration.

Gerald Fitzpatrick and Margaret-Anne Storey. 2017. The Risks of Good Enough Software Engineering. *IEEE Software* 34, 6 (2017), 14–19. doi:10.1109/MS.2017.4121224

Nicole Forsgren. 2021. DORA Metrics in Practice. *ACM Queue* (2021). https://queue.acm.org/

Nicole Forsgren, Jez Humble, and Gene Kim. 2018. Accelerate: The Science of Lean Software and DevOps. *IT Revolution* (2018).

Martin Fowler and Matthew Foemmel. 2006. Continuous Integration. In *ThoughtWorks.* https://martinfowler.com/articles/continuousIntegration.html

Robert~C. Martin. 2008. *Clean Code: A Handbook of Agile Software Craftsmanship.* Prentice Hall.

Weisong Shi, Jie Cao, Quan Zhang, Youhun Li, and Lanyu Xu. 2016. Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal* 3, 5 (2016), 637–646. doi:10.1109/JIOT.2016.2579198